

Assignment – 19.2

Name : Alli Harika

Batch : 21

Roll.No: 2303A510I7

Task 1 – Python to Java Conversion

Task:

Convert a Python program that performs file handling operations into an equivalent Java program using AI assistance.

Instructions:

- Prompt AI to translate a Python script that reads from and writes to a file into Java.
- Ensure proper handling of file streams, exceptions, and input/output formatting.
- Verify that the Java program produces the same file content and console output as the Python version.
- Test both implementations with sample input files.

Expected Output:

A correctly translated and executable Java program that performs the same file handling operations as the original Python script.

Prompt:

Convert the following Python file handling code into Java. Ensure proper file reading, writing, and exception handling.

Python Code:

with open("input.txt", "r") as file:

data = file.read()

with open("output.txt", "w") as file:

file.write(data.upper())

print("File copied and converted to uppercase.")

```
input.txt  J FileHandling.class X task1.py J FileHandling.java crud_operations.py BLACKBOX

1 // Source code is decompiled from a .class file using FernFlow decompiler (from IntelliJ IDEA)
2 import java.io.BufferedReader;
3 import java.io.BufferedWriter;
4 import java.io.FileReader;
5 import java.io.FileWriter;
6 import java.io.IOException;
7
8 public class FileHandling {
9     public FileHandling() {
10    }
11
12    public static void main(String[] var0) {
13        try {
14            BufferedReader var1 = new BufferedReader(new FileReader("input.txt"));
15            BufferedWriter var2 = new BufferedWriter(new FileWriter("output.txt"));
16
17            String var3;
18            while((var3 = var1.readLine()) != null) {
19                var2.write(var3.toUpperCase());
20                var2.newLine();
21            }
22
23            var1.close();
24            var2.close();
25            System.out.println("File copied and converted to uppercase.");
26        } catch (IOException var4) {
27            var4.printStackTrace();
28        }
29    }
30 }
31 }
```

```
19 > J FileHandling.java > FileHandling > main(String[])
1 import java.io.*;
2
3 public class FileHandling {
4     Run | Debug
5     public static void main(String[] args) {
6         try {
7             BufferedReader reader = new BufferedReader(new FileReader(fileName: "input.txt"));
8             BufferedWriter writer = new BufferedWriter(new FileWriter(fileName: "output.txt"));
9
10            String line;
11            while ((line = reader.readLine()) != null) {
12                writer.write(line.toUpperCase());
13                writer.newLine();
14            }
15
16            reader.close();
17            writer.close();
18
19            System.out.println(x: "File copied and converted to uppercase.");
20        } catch (IOException e) {
21            e.printStackTrace();
22        }
23    }
24 }
```

```
lab 19 > task1.py > ...
1  with open("input.txt", "r") as file:
2      data = file.read()
3
4  with open("output.txt", "w") as file:
5      file.write(data.upper())
6
7  print("File copied and converted to uppercase.")
```

```
lab 19 > input.txt
1  hello world
2  ai lab
```

```
lab 19 > output.txt
1  HELLO WORLD
2  AI LAB
3
```

```
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> python task1.py
File copied and converted to uppercase.
File copied and converted to uppercase.
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> javac FileHandling.java
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> javac FileHandling.java
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> java FileHandling
File copied and converted to uppercase.
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> █
```

Observation

- Python uses simple with open() syntax for file handling
- Java requires:
 - BufferedReader and BufferedWriter
 - Explicit exception handling (try-catch)
- Java code is more verbose compared to Python
- Both programs produce identical output files
- Python is more concise and easier for file operations

Task 2 – C++ to Python Function Conversion

Task:

Translate a C++ program that implements a sorting algorithm into Python using AI assistance.

Instructions:

- Provide AI with a C++ implementation of a sorting technique (e.g., Bubble Sort or Selection Sort).
- Ask AI to generate an equivalent Python implementation.
- Ensure logical correctness and validate results using multiple test cases.
- Confirm that the Python version maintains the same sorting behavior.

Expected Output:

A correct and executable Python program equivalent to the given C++ sorting implementation.

Prompt:

Convert the following C++ Bubble Sort program into Python while maintaining the same logic and output.

C++ Code:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int arr[5] = {5, 2, 9, 1, 3};
```

```

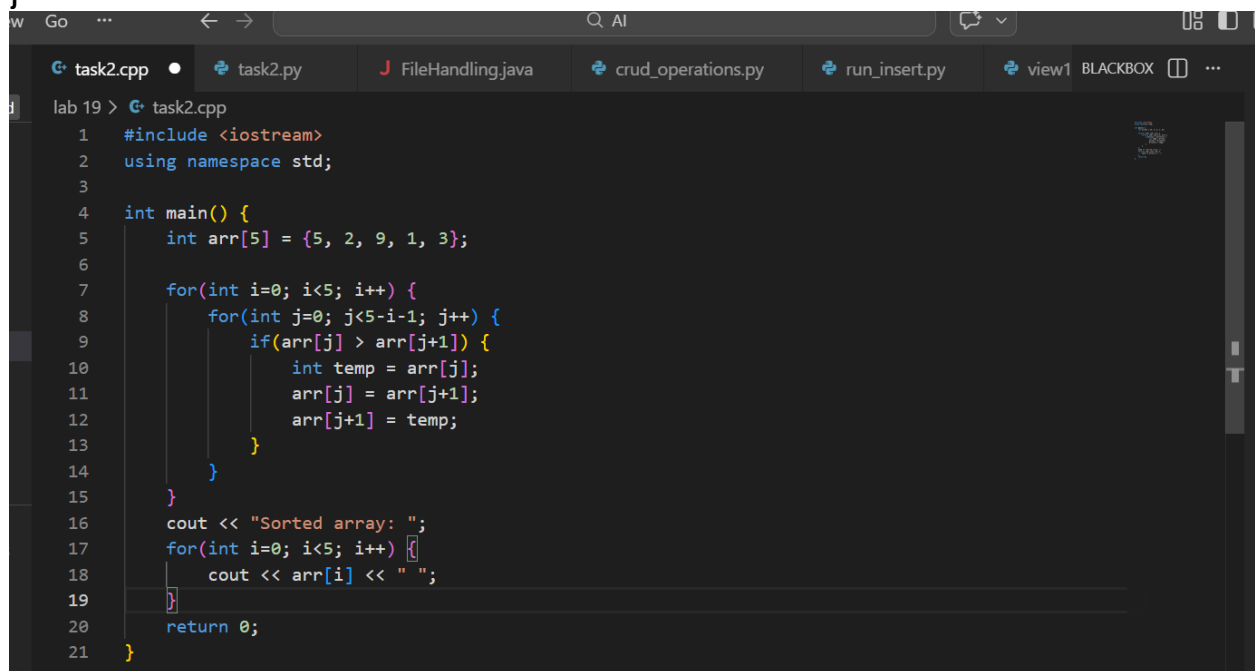
for(int i=0; i<5; i++) {
    for(int j=0; j<5-i-1; j++) {
        if(arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

```

```

for(int i=0; i<5; i++) {
    cout << arr[i] << " ";
}
return 0;
}

```



The screenshot shows a code editor with a dark theme. The top bar includes a search icon and the text 'AI'. Below the top bar, there are several tabs: 'task2.cpp' (active), 'task2.py', 'FileHandling.java', 'crud_operations.py', 'run_insert.py', 'view1', and 'BLACKBOX'. The main editor area displays the following C++ code:

```

lab 19 > task2.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int arr[5] = {5, 2, 9, 1, 3};
6
7      for(int i=0; i<5; i++) {
8          for(int j=0; j<5-i-1; j++) {
9              if(arr[j] > arr[j+1]) {
10                 int temp = arr[j];
11                 arr[j] = arr[j+1];
12                 arr[j+1] = temp;
13             }
14         }
15     }
16     cout << "Sorted array: ";
17     for(int i=0; i<5; i++) {
18         cout << arr[i] << " ";
19     }
20     return 0;
21 }

```

lab 19 > task2.py > ...

```
1  arr = [5, 2, 9, 1, 3]
2
3  for i in range(len(arr)):
4      for j in range(len(arr) - i - 1):
5          if arr[j] > arr[j + 1]:
6              arr[j], arr[j + 1] = arr[j + 1], arr[j]
7
8  print("Sorted array:", arr)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> .\task2.exe
Sorted array: 1 2 3 5 9
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> python task2.py
Sorted array: [1, 2, 3, 5, 9]
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> █
```

Observation

- Both implementations follow the same Bubble Sort logic
- Python uses lists instead of arrays
- Python swapping is simpler using tuple unpacking
- C++ requires more syntax (loops, variables, types)
- Output is logically identical in both languages
- Python code is shorter and more readable

Task 3 – SQL to Pandas Query

Task:

Convert a SQL query involving GROUP BY and aggregate functions into an equivalent Pandas Data Frame operation.

Instructions:

- Provide AI with a SQL query that calculates aggregate values (e.g., total sales per department).
- Prompt AI to generate equivalent Pandas code using groupby() and aggregation functions.
- Test the generated code using a sample dataset.
- Verify that both SQL and Pandas outputs match.

Expected Output:

An accurate Pandas Data Frame operation that produces results equivalent to the given SQL query with aggregation.

Prompt:

Convert the following SQL query into an equivalent Pandas DataFrame operation using `groupby()` and aggregation functions.

SQL Query:

SELECT department, SUM(salary)

FROM employees

GROUP BY department;

```
19 > task3.py > ...
1  import pandas as pd
2
3  # Sample dataset (like SQL table)
4  data = {
5      "department": ["HR", "IT", "HR", "IT", "Finance"],
6      "salary": [30000, 50000, 35000, 60000, 40000]
7  }
8
9  df = pd.DataFrame(data)
10
11 # Equivalent to SQL GROUP BY
12 result = df.groupby("department")["salary"].sum()
13
14 print(result)
```

```
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> python task3.py
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> python task3.py
department
Finance      40000
HR            65000
IT           110000
Name: salary, dtype: int64
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19>
```

Observation

- SQL uses declarative syntax (SELECT, GROUP BY)
- Pandas uses functional approach (`groupby()`, `.sum()`)
- Both produce the same aggregated result
- Pandas allows more flexibility for data analysis
- SQL is more structured and database-oriented
- Pandas integrates easily with Python workflows

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will select a practical real-world application requiring the same logic in two different programming languages.

Example:

- Converting a REST API implementation from Python (Flask) to Java (Spring Boot).
- Translating a data processing script from Python to C++ for performance optimization.

Instructions:

- Provide AI with source code in one programming language.
- Prompt AI to convert the code into another chosen language.
- Execute and test both versions to ensure identical functionality.
- Document differences observed in syntax, libraries, and execution behavior.

Expected Output:

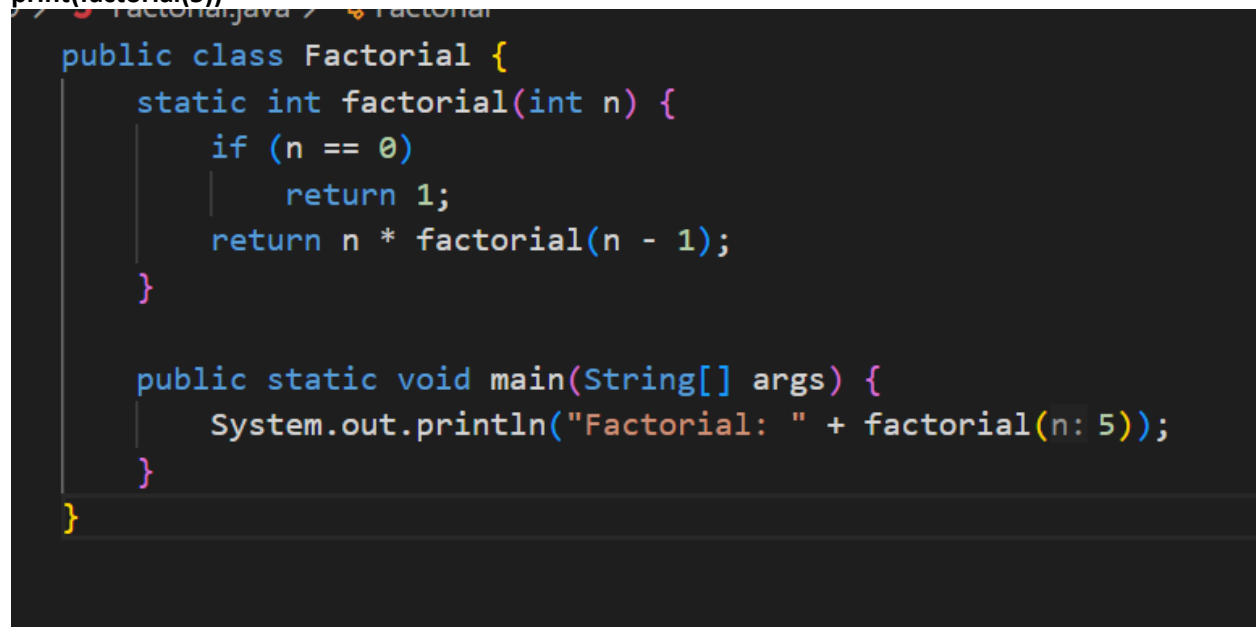
Functionally equivalent and executable code snippets implemented in two different programming languages, along with documented observations.

Prompt:

Convert the following Python factorial program into Java and ensure both versions produce the same output.

Python Code:

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)  
print(factorial(5))
```



```
Factorial.java  
public class Factorial {  
    static int factorial(int n) {  
        if (n == 0)  
            return 1;  
        return n * factorial(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Factorial: " + factorial(5));  
    }  
}
```



```
b 19 > task4.py > ...
1  def factorial(n):
2      if n == 0:
3          return 1
4      return n * factorial(n - 1)
5
6  print("Factorial:", factorial(5))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> python task4.py
Factorial: 120
PS C:\Users\dell\OneDrive\Desktop\AI\lab 19>
```

Observation

- Both programs use recursion
- Logic remains exactly the same
- Python is simple and concise
- Java requires:
 - Class definition
 - main() method
- Java is more structured but verbose
- Output is identical in both languages

Task -5: (Basic Function Translation (Python to C++))

Translate a simple Python function that checks whether a number is even or odd into C++ using AI assistance.

Instructions:

- Provide AI with a basic Python function that determines if a number is even or odd.
- Prompt AI to generate equivalent C++ code.
- Ensure correct syntax, input handling, and output formatting.
- Compile and run the C++ program to verify it produces the same result as the Python version.
- Test with at least three different input values.

Expected Output:

A simple and executable C++ program equivalent to the given Python

function, correctly identifying whether a number is even or odd for all test cases.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

Prompt:

Convert the following Python function into C++ with proper input handling and output formatting.

Python Code:

```
def check_even_odd(n):
```

```
    if n % 2 == 0:
```

```
        print("Even")
```

```
    else:
```

```
        print("Odd")
```

```
check_even_odd(5)
```

```

#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Enter a number: ";
    cin >> n;

    if(n % 2 == 0)
        cout << "Even";
    else
        cout << "Odd";

    return 0;
}

```

```

● PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> .\task5.exe
● Enter a number: 7
  Odd
● PS C:\Users\dell\OneDrive\Desktop\AI\lab 19> .\task5.exe
  Enter a number: 10
  Even
○ PS C:\Users\dell\OneDrive\Desktop\AI\lab 19>

```

Observation

- Logic ($n \% 2 == 0$) is same in both languages
- Python uses simple function call
- C++ requires:

- **main() function**
 - **Input using cin**
 - **Output using cout**
- **Python is shorter and easier**
- **C++ requires compilation before execution**
- **Both produce correct results for all test cases**